

### 5.1. Display Complete Process Memory Layout/Segments and Memory Addresses in C

**Aim:** To write a C program to demonstrate the typical process memory layout by displaying the memory addresses of variables and functions belonging to the Stack, Heap, BSS, Data, Read-Only Data, and Text/Code segments.

#### Algorithm

1. Start the program.
2. Declare initialized global and static variables to represent the Data segment.
3. Declare uninitialized global and static variables to represent the BSS segment.
4. Declare a string literal to demonstrate Read-Only Data.
5. Define two functions to demonstrate the Text/Code segment.
6. Inside the displayMemory() function:
  - o Declare local variables to represent the Stack.
  - o Dynamically allocate memory using malloc() to represent the Heap.
7. Store a value in the dynamically allocated memory.
8. Use the address operator & to obtain the addresses of variables.
9. Use %p with (void \*) to display the addresses of object variables.
10. Display the addresses of:
  - o Stack variables
  - o Heap memory
  - o BSS variables
  - o Data variables
  - o Read-only string literal
  - o Functions in the Text/Code segment
11. Release the dynamically allocated memory using free().
12. Stop the program.

#### Output:

---

---

#### PROCESS MEMORY LAYOUT

---

---

##### STACK

localInt : 0x7ffe7bf60204  
localFloat : 0x7ffe7bf60200  
localChar : 0x7ffe7bf601ff

##### HEAP

heapInt : 0x33ced2a0

##### BSS

uninitializedInt : 0x40404c  
uninitializedFloat : 0x404050  
staticUninitialized: 0x404054

##### DATA

globalInt : 0x404030  
globalFloat : 0x404034  
globalChar : 0x404038

##### RO-DATA

String Literal : 0x402008

##### TEXT / CODE

function1() : 0x401156  
function2() : 0x401167

## **5.2. Observing Stack Frames During Recursive Function Calls (Recursive Stack Analyzer)**

**Aim:** To write a C program to observe the stack frames created during recursive function calls by displaying the parameter value, parameter address, local variable value, local variable address, and recursive call number.

**Concept:** Every time a recursive function is called, a new stack frame (activation record) is typically created for that function call.

Each stack frame contains information such as:

- Function parameters
- Local variables
- Return information
- Other implementation-dependent data

In this experiment, the addresses of n and local Variable are displayed for every recursive call. This helps students observe that each recursive call has its own copy of parameters and local variables.

### **Algorithm**

1. Start the program.
2. Declare an integer variable n in the main() function.
3. Read the value of n from the user.
4. Check whether n is negative.
5. If n is negative, display "Invalid number" and stop.
6. Otherwise, call the recursive function: factorial(n, 1);
7. In the factorial() function:
  - o Receive n and call as parameters.
  - o Create a local variable localVariable.
  - o Display the current recursive call number.
  - o Display the value and address of parameter n.
  - o Display the value and address of localVariable.
8. Check whether  $n \leq 1$ .
9. If  $n \leq 1$ , return 1 and terminate the current recursive call.
10. Otherwise, call: factorial(n - 1, call + 1)
11. Repeat Steps 7–10 until the base condition is reached.
12. As recursive calls return, calculate the factorial.
13. Display the final factorial result.
14. Stop the program.

### **Output:**

Enter number: 5

```
-----  
Call Number      : 1  
Parameter n      : 5  
Parameter Address : 0x7ffdeef3436c  
Local Variable   : 50  
Local Address    : 0x7ffdeef3437c
```

```
-----  
Call Number      : 2  
Parameter n      : 4  
Parameter Address : 0x7ffdeef3432c  
Local Variable   : 40  
Local Address    : 0x7ffdeef3433c
```

-----  
Call Number : 3  
Parameter n : 3  
Parameter Address : 0x7ffdeef342ec  
Local Variable : 30  
Local Address : 0x7ffdeef342fc

-----  
Call Number : 4  
Parameter n : 2  
Parameter Address : 0x7ffdeef342ac  
Local Variable : 20  
Local Address : 0x7ffdeef342bc

-----  
Call Number : 5  
Parameter n : 1  
Parameter Address : 0x7ffdeef3426c  
Local Variable : 10  
Local Address : 0x7ffdeef3427c

FACTORIAL = 120

### 5.3. Heap Memory Lifecycle and Address Tracking

**Aim:** To write a C program to demonstrate the complete lifecycle of dynamically allocated heap memory using malloc(), initialization, usage, realloc(), free(), and NULL, while displaying the value and memory address of every array element.

**Concept:** Dynamic memory is allocated from the heap during program execution.

The lifecycle is:

malloc()



Initialize



Use



realloc()



Use



free()



ptr = NULL

The experiment also demonstrates that each array element has a different memory address.

#### Algorithm

1. Start the program.
2. Declare an integer pointer ptr.
3. Allocate memory for 3 integers using malloc().
4. Check whether memory allocation was successful.
5. Display the address of each allocated element.
6. Initialize the three elements with values 10, 20, and 30.
7. Display the value and address of each element.
8. Use the allocated memory and display all elements.
9. Increase the memory size from 3 integers to 5 integers using realloc().
10. Check whether reallocation was successful.
11. Display the values and addresses of the existing elements.
12. Initialize the newly allocated elements with 40 and 50.
13. Display the value and address of all five elements.
14. Release the allocated memory using free().
15. Set the pointer to NULL.
16. Display the value of the pointer.
17. Stop the program.

#### Output:

After malloc():

ptr[0] Address = 0x17b4d2a0

ptr[1] Address = 0x17b4d2a4

ptr[2] Address = 0x17b4d2a8

After initialization:

ptr[0] = 10, Address = 0x17b4d2a0

ptr[1] = 20, Address = 0x17b4d2a4

ptr[2] = 30, Address = 0x17b4d2a8

Using memory:

Value = 10, Address = 0x17b4d2a0

Value = 20, Address = 0x17b4d2a4

Value = 30, Address = 0x17b4d2a8

After realloc():

ptr[0] = 10, Address = 0x17b4d2a0

ptr[1] = 20, Address = 0x17b4d2a4

ptr[2] = 30, Address = 0x17b4d2a8

After adding new elements:

ptr[0] = 10, Address = 0x17b4d2a0

ptr[1] = 20, Address = 0x17b4d2a4

ptr[2] = 30, Address = 0x17b4d2a8

ptr[3] = 40, Address = 0x17b4d2ac

ptr[4] = 50, Address = 0x17b4d2b0

After free():

Heap memory released.

After ptr = NULL:

ptr = (nil)